Lidar Following

For instructions on app connection, please consult the tutorial located in "1.Getting Ready/ 1.4 APP Control".

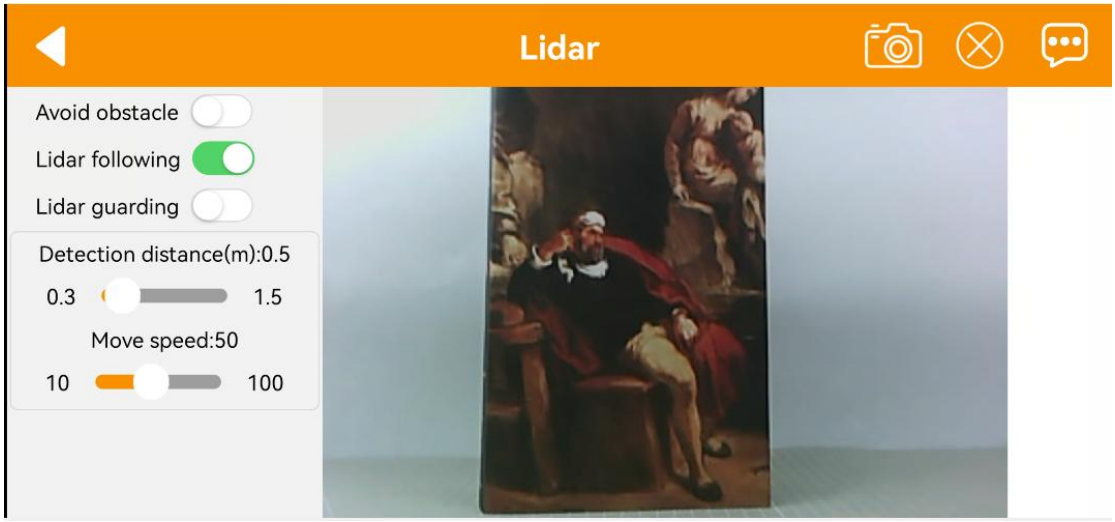
1. Enable Game

1.1 Initiate Lidar Game through App

1) Connect the robot to "WondePi" app.

2) Click-on Lidar to navigate to the game interface.

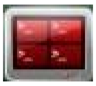
3) Switch on "Lidar following" button to start the game.



1.2 Initiate Lidar Game Using Command

1) Start the robot, and access the robot system desktop using VNC according

to the tutorial saved in "8.Set Development Environment/Lesson 1 VNC Installation and Connection".

2) Click-on  to open the command line terminal.

3) Execute the command and press Enter to deactivate the automatic start service for the app.

~/.stop_ros.sh

```
❯ ~/.stop_ros.sh
zsh: killed     ~/.stop_ros.sh
```

4) Run the command to enable the local services for app-related game and chassis control services.

ros2 launch app lidar_node.launch.py debug:=true

```
❯ ~/.stop_ros.sh
zsh: killed     ~/.stop_ros.sh
                                                    KILL ✘ | 09:53:18
 ros2 launch app lidar_node.launch.py debug:=true
```

5) Open a new terminal, and execute the command. Then, press Enter to start the Lidar game.

ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}

```
❯ ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}
waiting for service to become available...
requester: making request: std_srvs.srv.Trigger_Request()

response:
std_srvs.srv.Trigger_Response(success=True, message='enter')
```

Note: The robot's performance initiated by the app and command is identical.

6) Enter the command and hit Enter to launch Lidar following game.

ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 2}"

```
❯ ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 2}"
waiting for service to become available...
requester: making request: interfaces.srv.SetInt64_Request(data=2)

response:
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

7) If you want to exit the game, enter the command and press "Enter".

ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data:

```
0}"
```

```
❯ ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 0}"
requester: making request: interfaces.srv.SetInt64_Request(data=0)

response:
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

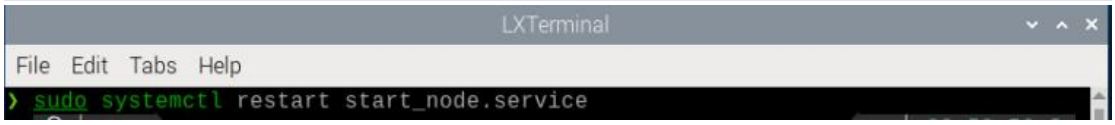
Note: The game will continue to run under the current Raspberry Pi power-on state if not exited. To avoid excessive use of the Raspberry Pi's operating memory, please follow the above instructions to close the current game before executing other games.

8) If you need to terminate the program, press short-cut “Ctrl+C” on the terminal opened in steps 4) and 5).

After experiencing the Lidar game, you can activate the app service either by using a command or restarting the robot. If the app service is not activated, related app functions will be disabled. In the case of a robot restart, the app service will start automatically.

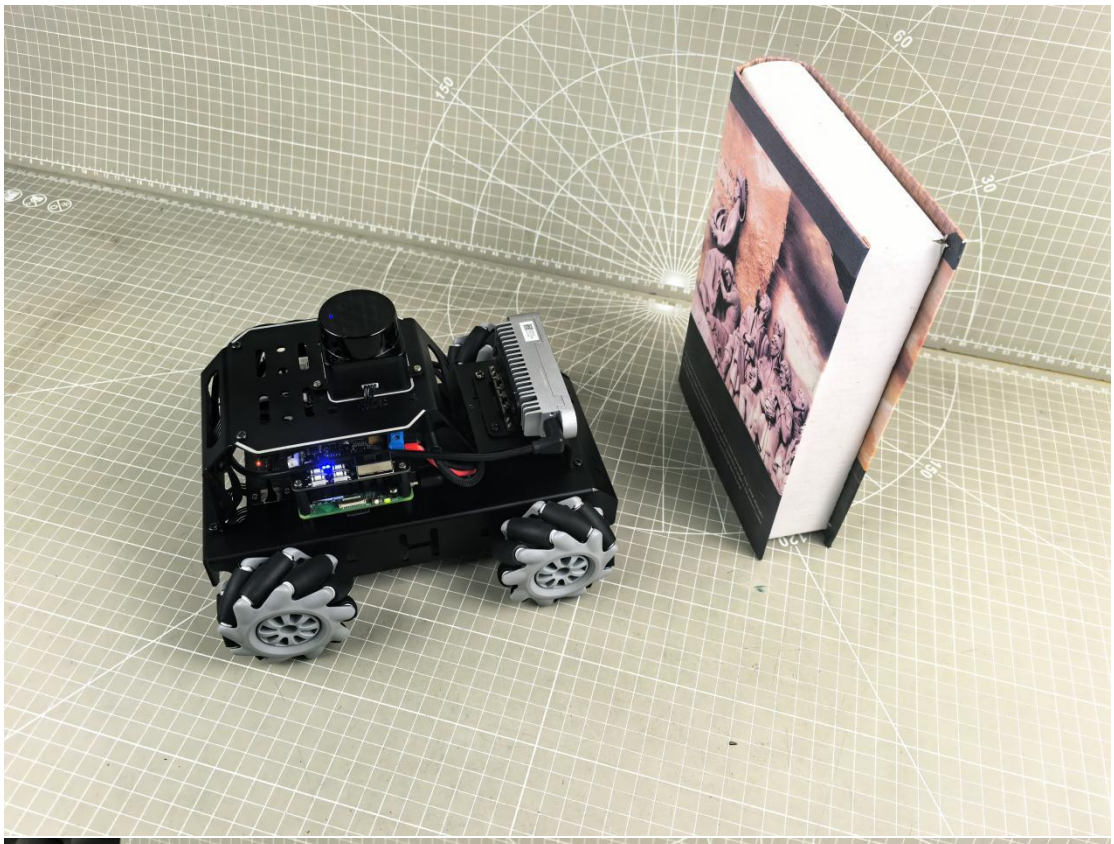
Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

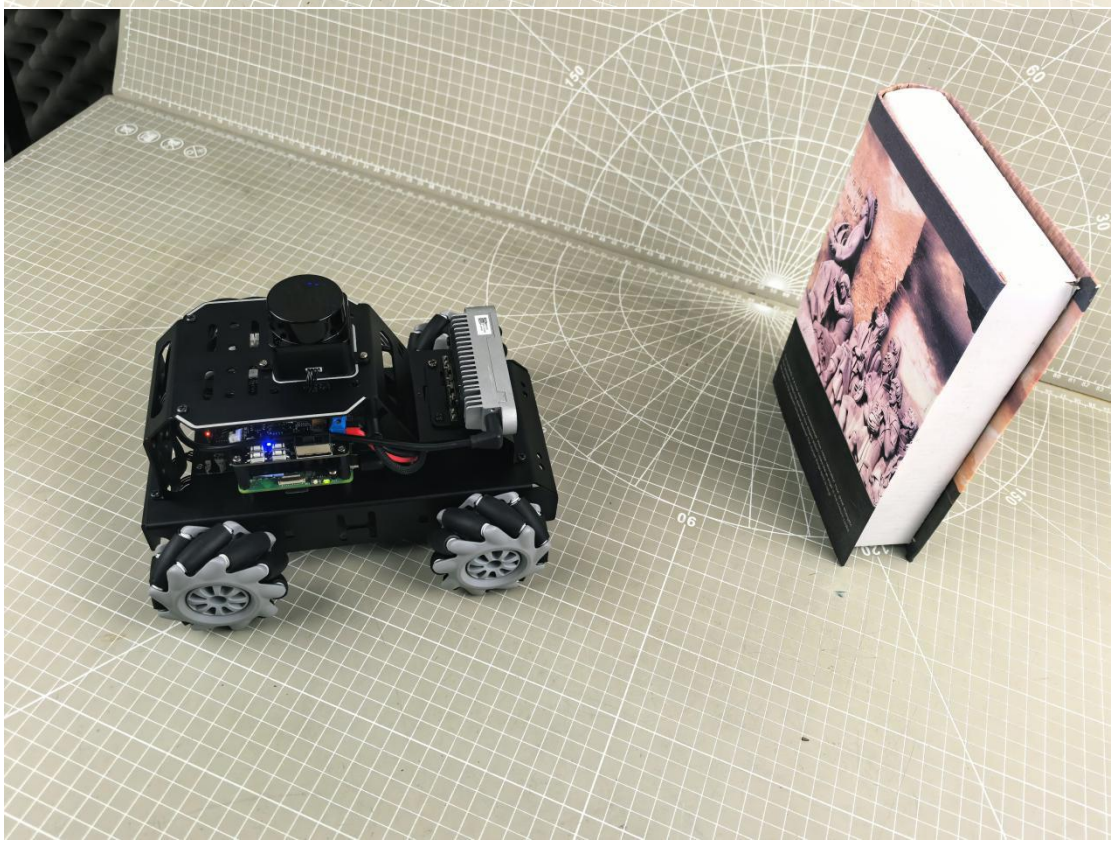
```
sudo systemctl restart start_node.service
```



2. Program Outcome

Let's use a book as the object to be detected. When using the Lidar following function, it's important to ensure that the object to be detected is higher than the scanning height of the Lidar mounted on the MentorPi. This will enable the Lidar to effectively scan the position information of the object. Once the position information is obtained, the MentorPi will adjust its position to maintain a distance of around 0.35m between itself and the obstacle.



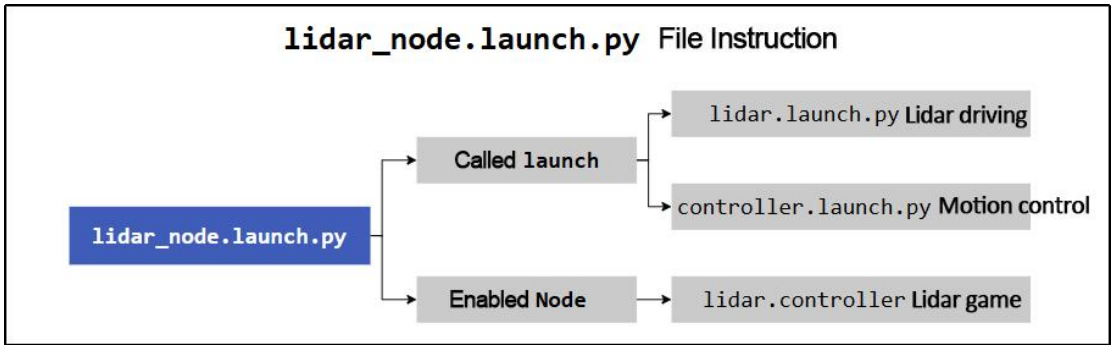


3. Program Analysis

3.1 Launch File

The launch file is located in:

/home/ubuntu/ros2_ws/src/app/launch/lidar_node.launch.py




```python
import os

from launch_ros.actions import Node
from launch.conditions import IfCondition
from launch import LaunchDescription, LaunchService
from launch.substitutions import LaunchConfiguration
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import IncludeLaunchDescription, OpaqueFunction, GroupAction, DeclareLaunchArgument

def launch_setup(context):
    compiled = os.environ['need_compile']
    debug = LaunchConfiguration('debug', default='false')
    debug_arg = DeclareLaunchArgument('debug', default_value=debug)
    if compiled == 'True':
        controller_package_path = get_package_share_directory('controller')
        peripherals_package_path = get_package_share_directory('peripherals')
    else:
        controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
        peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'

    lidar_controller_node = GroupAction([
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
            condition=IfCondition(debug),
            ),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(controller_package_path, 'launch/controller.launch.py')),
            condition=IfCondition(debug),
            ),

        Node(
            package='app',
            executable='lidar_controller',
            output='screen',
            ),
```

- **Set the storage path**

Retrieve the paths for the two packages: peripherals and controller.

```python
    if compiled == 'True':
        controller_package_path = get_package_share_directory('controller')
        peripherals_package_path = get_package_share_directory('peripherals')
    else:
        controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
        peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
```

- **Initiate other Launch files**

```
lidar_controller_node = GroupAction([
    IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
        condition=IfCondition(debug),
        ),

    IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(controller_package_path, 'launch/controller.launch.py')),
        condition=IfCondition(debug),
        ),
```

lidar.launch.py Lidar launch

controller.launch.py Motion control launch

- **Initiate Node**

```
        Node(
            package='app',
            executable='lidar_controller',
            output='screen',
            ),
    ])
```

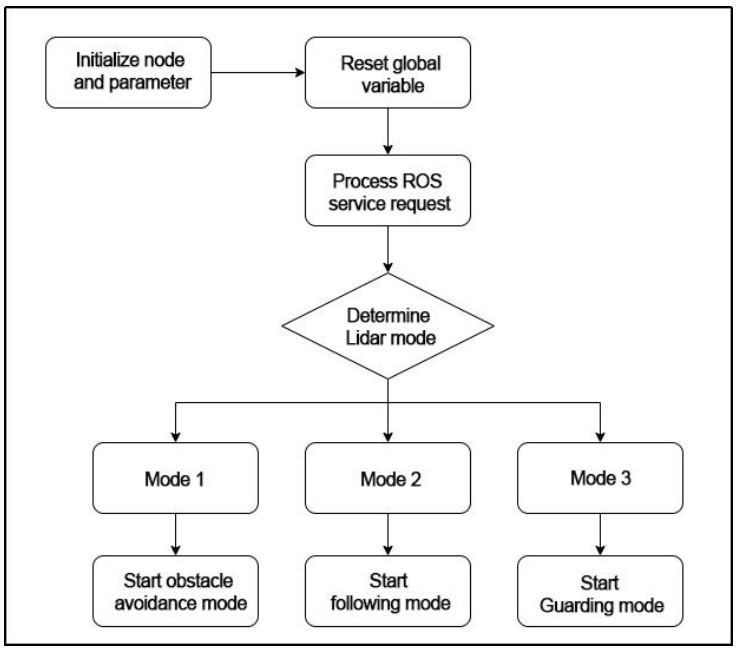
Enable Lidar game node.

3.2 Source Code File

The source code of the program is saved in:

/home/ubuntu/ros2_ws/src/app/app/lidar_controller.py

The process logic of the program based on the implemented effect is shown in the diagram:



◆ **Initialization**

```
class LidarController(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)

        self.name = name
        self.running_mode = 0
        self.threshold = 0.6  # meters
        self.scan_angle = math.radians(360)  # radians
        self.speed = 0.2
        self.last_act = 0
        self.timestamp = 0
        self.angle_data = []
```

self.name: stores the incoming name as the property of the object.

Other attributes include parameters for controlling the Lidar, such as operating mode, threshold, scanning angle, and speed.

self.lock: creates a thread lock for safe access in a multi-threaded environment.

◆ **Create ROS service**

```
        self.create_service(Trigger, '~/enter', self.enter_srv_callback)  # 进入玩法(enter the game)
        self.create_service(Trigger, '~/exit', self.exit_srv_callback)  # 退出玩法(exit the game)
        self.create_service(SetInt64, '~/set_running', self.set_running_srv_callback)  # 开启玩法(start the game)
        Heart(self, self.name + '/heartbeat', 5, lambda _: self.exit_srv_callback(request=Trigger.Request(), response=Trigger.Response()))  # 心跳包(heartbeat package)
        self.create_service(SetFloat64List, '~/set_param', self.set_parameters_srv_callback)  # 参数设置(set parameter)
```

Enter game: A ROS service named "/enter" is created with the type Trigger, and the callback function is enter_srv_callback. When this service is called, it will execute the enter_srv_callback function.

Exit game: A ROS service named "/exit" is created with the type Trigger, and the callback function is exit_srv_callback. When this service is called, it will execute the exit_srv_callback function.

Enable game: A ROS service named "/set_running" is created with the type SetInt64, and the callback function is set_running_srv_callback. When this service is called, it will execute the set_running_srv_callback function.

Set parameters: A ROS service named "/set_param" is created with the type SetFloat64List, and the callback function is set_parameters_srv_callback. When this service is called, it will execute the set_parameters_srv_callback function.

◆ **Lidar following**

```
                # 拼合距离数据，从右半侧逆时针到左半侧(the merged distance data from right half counterclockwise to the left half)
                ranges = np.append(right_range[::-1], left_range)
                self.get_logger().info(str(ranges))
                nonzero = ranges.nonzero()
                nonan = np.isfinite(ranges[nonzero])
                dist_ = ranges[nonzero][nonan]
                if len(dist_) > 0:
                    dist = dist_.min()
                    min_index = list(ranges).index(dist)
                    angle = -angle + lidar_data.angle_increment * min_index  # 计算最小值对应的角度(calculate the angle corresponding to the minimum value)
                    # self.get_logger().info(str(dist_))
                    # self.get_logger().info(str([dist, angle]))
                    if dist < self.threshold and abs(math.degrees(angle)) > 5:  # 控制左右(control the left and right)
                        if self.lidar_type != 'G4':
                            self.pid_yaw.update(-angle)
                            twist.angular.z = common.set_range(self.pid_yaw.output, -self.speed * 6.0, self.speed * 6.0)
                        elif self.lidar_type == 'G4':
                            self.pid_yaw.update(angle)
                            twist.angular.z = -common.set_range(self.pid_yaw.output, -self.speed * 6.0, self.speed * 6.0)
                    else:
                        self.pid_yaw.clear()

                    if dist < self.threshold and abs(0.2 - dist) > 0.02:  # 控制前后(control the front and back)
                        self.pid_dist.update(self.threshold / 2 - dist)
                        twist.linear.x = common.set_range(self.pid_dist.output, -self.speed, self.speed)
                    else:
                        self.pid_dist.clear()
                    if abs(twist.angular.z) < 0.008:
                        twist.angular.z = 0.0
                    if abs(twist.linear.x) < 0.05:
                        twist.linear.x = 0.0
                    self.mecanum_pub.publish(twist)
```

The code concatenates the distance data from the left and right sides of the robot, obtained from the Lidar, into an array “ranges” to form a circular scan. It then calculates the minimum distance value “dist” and its corresponding angle “angle” from the “ranges”.

The “dist” is checked if it is less than a threshold value. If it is and the angle deviates from the forward direction by more than 5 degrees, the robot will rotate to avoid the obstacle. The angular velocity is controlled using a PID controller. If “dist” is less than the threshold and the robot is close to a distance of 0.2m in the forward direction, the robot will stop or move forward using a PID controller for the linear velocity. The control commands are published as a Twist message to the robot to execute the desired actions.